

CS339: Abstractions and Paradigms for Programming

Logic Paradigm

Manas Thakur
CSE, IIT Bombay



Autumn 2025

Declarative vs Operational

➤ Square root of x:

- A number y such that $y^2 = x$ Declarative
- Newton's method

Operational

```
(define (sqrt x)
  (define (good-enough? guess)
    (< (abs (- (square guess) x)) 0.001))
  (define (improve guess)
    (average guess (/ x guess)))
  (define (sqrt-iter guess)
    (if (good-enough? guess)
        guess
        (sqrt-iter (improve guess))))
  (sqrt-iter 1.0))
```



Sum a list of numbers: Imperative (Java)

```
int sum(int[] list) {  
    int result = 0;  
    for (int i = 0; i < list.length; i++) {  
        result += list[i];  
    }  
    return result;  
}
```



Sum a list of numbers: Functional (Scheme)

```
(define (sum list)
  (if (null? list)
      0
      (+ (car list) (sum (cdr list)))))
```



Sum a list of numbers: Logic (Prolog)

```
sum([], 0).  
sum([H | T], N) :- sum(T, M), N is H + M.
```

Functional is sometimes close to declarative, but logic is closer.



Logic Paradigm

- Programs consist of just facts and rules.
- Not necessary to describe the “procedure” or the control flow at a very low-level.
- Who does the computation then?
- In other words, who has the onus of translating the “declarative” description to an “algorithm” that computes on the von-Neumann architecture?
 - The Interpreter!



Logic Paradigm: Usage

- **Prolog** quite popular in rule-based Artificial Intelligence.
- **Datalog** becoming very popular in program analysis, code optimization, and type inference.
- **SQL** already the de-facto of relational databases.



Predicates and Horn Clauses

- If it is precipitating in a city C and the temperature in C is freezing, then it is snowing in C.

- snowing(C); precipitation(C); freezing(C)

Predicates

- snowing(C) <- precipitation(C) AND freezing(C)

Horn clauses

- When does it snow at Mumbai?
 - **Instantiate** the variables.







House: STARK



RICKARD
STARK



UNKNOWN



BRANDON
STARK



EDDARD
STARK



CATELYN
TULLY



RHAEGAR
TARGARYEN



LYANNA
STARK



BENJEN
STARK



ROBB
STARK



SANSA
STARK



ARYA
STARK



BRAN
STARK



RICKON
STARK



JON
SNOW

Toxic
Chemistry ©

(The Shortest?) Introduction to Prolog

➤ Two kinds of **terms**:

➤ Facts

- `father(ned, arya).`
- `mother(catelyn, bran).`

➤ Rules

- `parent(X, Y) :- father(X, Y).`
- `parent(X, Y) :- mother(X, Y).`
- `grandparent(X, Z) :- parent(X, Y), parent(Y, Z).`

➤ Querying:

- `?- father(ned, sansa).`
- `?- grandparent(rickard, Who).`

Rules of the game (aka Syntax):

- Constants start with small letters.
- Variables start with capital letters.
- Full stop necessary after each fact/rule.
- No space before the opening parenthesis.
- Multiple terms with the same *head* indicate disjunction.
- A comma between terms indicates conjunction.



How does the Prolog engine work (*aka* semantics)?

- **Program = logic + control.**
 - Logic is specified by user; control is managed by runtime.
- Given a query:
 - Consult the facts and rules in **top-down** order.
 - Try to **instantiate** variables in the RHS of rules.
 - Report instantiated values that **satisfy** the predicates resultant from the query.



Resolution and Unification

- **Resolution.** If h is the head of a Horn clause and it matches with one of the terms of another Horn clause, then that term can be replaced by h .
- **Unification.** A pattern-matching process that determines what particular instantiations can be made to variables while making a series of *simultaneous* resolutions.
 - Which resolutions are simultaneous?
 - Those that satisfy the given set of predicates.
- Example: ?- parent(brandon, bran).



Searching and Backtracking

*When the path ahead is not nice,
say even mice, that backtracking is wise.*

- Basic idea of logic paradigm:
 - **Search** through the solution space while trying to unify variables with values, till you get a solution.
 - If no further resolution can be done, then **backtrack** and try a different instantiation.
- Example: ?- parent(brandon, bran).
- **Observe:** Multiple solutions are possible.
 - Example: ?- grandparent(rickard, Whoa11).



Who lives where

- Adarsh, Kunj, Malay, Niral and Priyanshu live in a five-room corridor in the H17 hostel, starting from 1 to 5.
- Adarsh doesn't live in the fifth room and Kunj doesn't live in the first.
- Malay doesn't live in the first or the last rooms, and he is not in a room adjacent to Priyanshu or Kunj.
- Niral lives in some room numbered after that of Priyanshu.
- Who lives in which room?



Who lives where (Cont.)

- Say the solution should look like this:
 - `[room(_,5), room(_,4), room(_,3), room(_,2), room(_,1)]`
- The five variables to be assigned rooms are: A, K, M, N and P.
- Say the structure `rooms(Rooms)` contains the solution list.
- The first constraint can be encoded as:
 - `member(room(adarsh, A), Rooms), A \= 5.`
- We can similarly add the other constraints to give the complete *program*.



Who lives where (Cont.)

```
rooms([room(_,5),room(_,4),room(_,3),room(_,2),room(_,1)]).
hostel(Rooms) :- rooms(Rooms),
    member(room(adarsh, A), Rooms), A \= 5,
    member(room(kunj, K), Rooms), K \= 1,
    member(room(malay, M), Rooms), M \= 1, M \= 5,
    member(room(priyanshu, P), Rooms),
    not(adjacent(M, P)), not(adjacent(M, K)),
    member(room(niral, N), Rooms), N > K,
    print_rooms(Rooms).
```

where `adjacent` and `print_rooms` are defined as follows:

```
adjacent(X, Y) :- X == Y+1.
adjacent(X, Y) :- X == Y-1.
print_rooms([A | B]) :- write(A), nl, print_rooms(B).
print_rooms([]).
```

Solution: `?- hostel(X).`

Yes, logic paradigm began the times when computers started replacing humans!



Thursday's is the last class

- We would
 - talk about other paradigms that exist;
 - discuss a few key learnings of the course;
 - and answer questions collected through an AMA form:

